MongoDB

CRUD Operations

Type		Syntax
Database	Show Databases	show dbs
	Switch to a Database	use databaseName
	Create a Database (Switching to a non-existent database after creating)	use newDatabase
	Drop complete Database	db.dropDatabase()
Collections	Create a Collection	db.createCollection("collectionName")
	Read (Show) Collections	show collections
	Update Collections	db.oldcollectionname.renameCollection("newcollectionname")
	Drop a Collection	db.collectionName.drop()
Document	Insert a Document (one by one)	db.collectionName.insertOne({ key: "value" })
	Insert a Document (Multiple Entry)	db.collectionName.insertMany([{ key1: "value1" },{ key2: "valu2e" }…..])
	Find All Documents in a Collection	db.collectionName.find()
	Find Documents with a Query	db.collectionName.find({ key: "value" })
	Find Documents with a Query & Projection	db.collectionName.find({ key: "value" },{projection key:Boolean value}) *Always mention _id:0 in projection in case want to exclude
	Update a Document (One by One)	db.collectionName.updateOne({ key: "value" }, { $set: { key: "new_value" } })
	Update a Document (Multiple)	db.collectionName.updateMany({ key: "value" }, { $set: { key: "new_value" } })
	Remove a Document	db.collectionName.deleteOne({ key: "value" }) db.collectionName.deleteMany({ key: "value" }) *We can remove documents by using the remove method.

Logical Operator

Operator	Description
$and	It is used to join query clauses with a logical AND and return all documents that match the given conditions of both clauses. Syntax: db.collectionName.find({$and: [{key:value},..]}) db.student.find({**$and**:[{name:"P1"},{age:28}]}) **or** db.student.find({name:"P1",age:28})
$or	It is used to join query clauses with a logical OR and return all documents that match the given conditions of either clause. Syntax: db.collectionName.find({$or: [{key:value},..]}) db.student.find({**$or**:[{name:"P1"},{age:28},..]})
$nor	It is used to join query clauses with a logical NOR and return all documents that fail to match both clauses. db.collectionName.find({$nor: [{key:value},..]}) db.people.find({**$nor**: [{name: "P1"},{age:20}]})
$not	It is used to invert the effect of the query expressions and return documents that does not match the query expression. Syntax:db.collectionName.find ({ keyfield: { $not: { **operator:value**} } } • Always comes with another **operator and Value** db.student.find({ age: { $not: { $eq: 18 } } }) *Always comes with Conditional operator

Comparison Operators

Operator	Description
$eq	Matches values that are equal to the given value. **Syntax: { field: { $eq: value } }** db.people.find({age:{$eq:20}})
$gt	Matches if values are greater than the given value. **Syntax: { field: { $gt: value } }** db.people.find({age:{$gt:20}})
$lt	Matches if values are less than the given value. **Syntax: { field: { $lt: value } }** db.people.find({age:{$lt:20}})
$gte	Matches if values are greater or equal to the given value. **Syntax: { field: { $gte: value } }** db.people.find({age:{$gte:20}})
$lte	Matches if values are less or equal to the given value. **Syntax: { field: { $lte: value } }** db.people.find({age:{$lte:20}})
$in	Matches any of the values in an array. **Syntax: { field: { $nin: [<value1>, <value2>, ... <valueN>] } }** db.people.find({age:{$in:[45,70]}})
$ne	Matches values that are not equal to the given value. **Syntax: { field: { $ne: value } }** db.people.find({age:{$ne:20}})
$nin	Matches none of the values specified in an array. **Syntax: { field: { $nin: [<value1>, <value2>, ... <valueN>] } }** db.people.find({age:{$nin:[45,70]}})

Methods

Method	Syntax	Description	Key Notes
.findOne()	db.collection.findOne({ query })	Returns **only the first** document that matches the query criteria.	Returns a single document object (not a cursor). Returns null if no match is found.
.limit()	db.collection.find({}).limit(x)	Restricts the result set to a maximum of x documents.	Used commonly for pagination. x must be a positive integer.
.skip()	db.collection.find({}).skip(x)	Skips the first x documents in the query results.	Controls where MongoDB begins returning results.
.sort()	db.collection.find({}).sort({ field: 1 })	Sorts the documents in the result set by the specified field(s).	Use 1 for **ascending** (A-Z, 0-9) and -1 for **descending** (Z-A, 9-0).
.count()	db.collection.find({}).count()	Returns the total count of documents that match the query.	**Legacy:** Deprecated in modern MongoDB drivers. Use countDocuments instead.
.countDocuments()	db.collection.countDocuments({ query })	Returns the count of documents matching the query.	**Modern Standard:** Accurate, supports aggregation, and is standard in Mongoose.

Field Update operators & Options

Syntax

db.collection.updateOne(filter,document,options)

Updates only one document where filter is matched

db.collection.updateMany(filter,document,options)

Updates only all documents where filter is matched

Parameters:

1. **filter**: The selection criteria for the update, same as **find()** method.
2. **document**: A document or pipeline that contains modifications to apply.
3. **options**: Optional. May contains options for update behavior. It includes upsert, writeConcern, collation, etc.
   {upsert:true} ----------→ Update & insert

Document operators in update

Name	Description
$inc	Increments the value of the field by the specified amount. **{ $inc: { <field1>: <amount1>, <field2>: <amount2>, ... } }** Examples: 1. db.PKP.updateMany({},{$inc:{age:10}}); Increase age field values by 10 for all documents 2. db.PKP.updateOne({},{$inc:{age:15}}); Increase age field values by 15 of 1st document 3. db.PKP.updateOne({},{$inc:{age:-15}}); Increase age field values by (-15) of 1st document
$mul	Multiplies the value of the field by the specified amount. **{ $mul: { <field1>: <number1>, ... } }** The field to update must contain a numeric value. 1. db.PKP.updateOne({},{$mul:{age:15}}); Multiply age field by 15 of 1st document

	2. db.PKP.updateMany({name:'N1'},{$mul:{age:0.5}}); Multiply age field by 0.5 for all documents where name is "N1" 3. db.PKP.updateMany({},{$mul:{age:2}}); Multiply age field by 2 for all documents
$rename	Renames a field. **{$rename: { <field1>: <newName1>, <field2>: <newName2>, ... } }** 1. db.PKP.updateMany({},{$rename:{'name':'uname'}}) 2. db.PKP.updateMany({},{$rename:{'name':'Uname','branch':'Branch'}})
$set	Sets the value of a field in a document. **syntax:{ $set: { <field1>: "", ... } }** Example: db.PKP.updateOne({age:{$eq:21}},{$set:{branch:"CSE"}})
$unset	Removes the specified field from a document. **syntax:{ $unset: { <field1>: "", ... } }** Example: db.PKP.updateOne({age:{$eq:21}},{$unset:{branch:"CSE",age: 21}}) **Note:** The values for each field can be "" or 1 . it doesn't matter. Here, fields which are mentioned will be removed where the age is 21.
$currentDate	Sets the value of a field to current date, either as a Date or a Timestamp. **{ $currentDate: { <field1>: <typeSpecification1>, ... } }** Example: db.PKP.updateOne({name: "ABC"}, {$currentDate: {Date: true}})

Note:

Update will only update the values if the document is already exist with mentioned filter value.

If document does not exist in collection with mentioned filter value and we want to add document with the mentioned fields then we have to add 3rd parameter **upsert:true.**

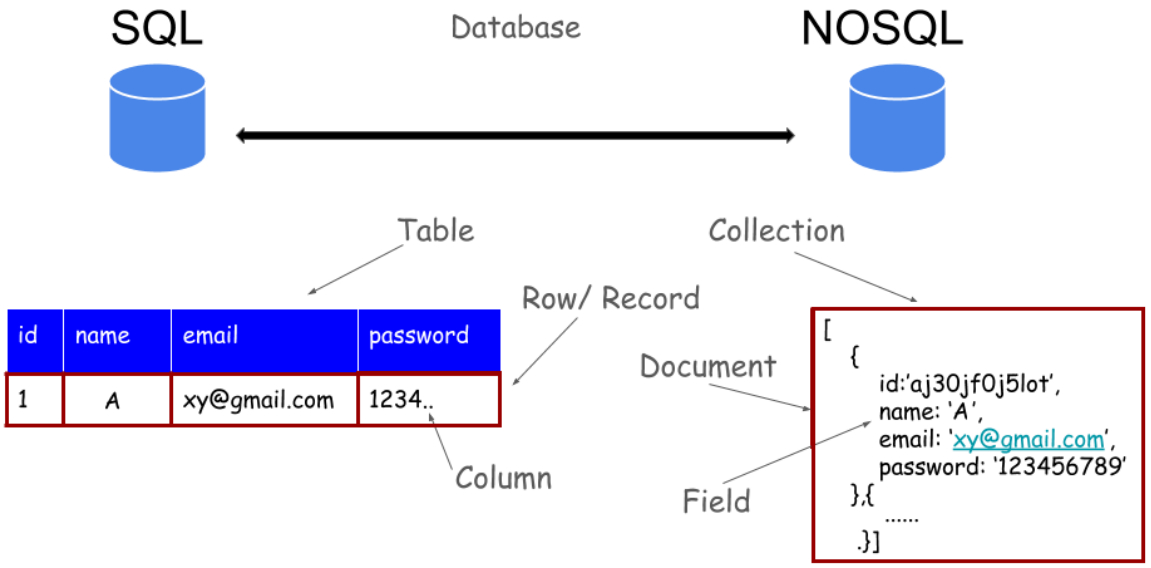
Regular Expression [$Regex]

You can use any of these equivalent syntaxes for standard matching:

- { field: { $regex: /pattern/ } }
- { field: { $regex: 'pattern' } }
- { field: /pattern/ } *(Most common/shorthand)*

Case	Objective	MongoDB Query Syntax	Regex Breakdown
1	**Substring Match** *(Finds "patel" anywhere in the string)*	db.PKP.find({ name: /patel/ })	/patel/ matches the literal characters anywhere.
2	**Case-Insensitive Match** *(Finds "Patel", "PATEL", "patel", etc.)*	db.PKP.find({ name: /patel/i })	i is the case-insensitive modifier flag.
3	**Starts With***(Matches strings beginning with "patel")*	db.PKP.find({ name: /^patel/ })	^ asserts the **start** of the string.
4	**Ends With***(Matches strings ending with "patel")*	db.PKP.find({ name: /patel$/ })	$ asserts the **end** of the string.
5	**Ends with Digit** *(Ends in 0-9)*	db.PKP.find({ name: /[0-9]$/ })	[0-9] matches any single digit at the end ($).
6	**Starts with Digit** *(Starts with 0-9)*	db.PKP.find({ name: /^[0-9]/ }) ordb.PKP.find({ name: /^\d/ })	^ starts with; [0-9] or \d represents any digit.
7	**Strictly Digits Only** *(At least 1 digit, no letters/spaces)*	db.PKP.find({ name: /^[0-9]+$/ })	+ requires **one or more** characters.
8	**Strictly Digits (Or Empty)***(Numbers only, but allows empty strings)*	db.PKP.find({ name: /^[0-9]*$/ })	* matches **zero or more** times.
9	**Specific Character Length** *(Matches names of exactly 3 to 10 letters)*	db.PKP.find({ name: /^[A-Za-z]{3,10}$/ })	[A-Za-z] limits to alphabets. {3,10} sets min/max length.

SQL NOSQL Mapping



SQL	MongoDB
Create and Alter commands	
CREATE TABLE PKP (id MEDIUMINT NOT NULL AUTO_INCREMENT, user_id Varchar(20), PRIMARY KEY (id))	db.createCollection ("PKP") ** creates collection no need to define schema
ALTER TABLE PKP ADD join_date DATETIME	db.PKP.updateMany({ }, { $set: { join_date: new Date() } })
ALTER TABLE PKP DROP COLUMN join_date	db.PKP.updateMany({ }, { $unset: { "join_date": "" } })
DROP TABLE PKP	db.PKP.drop ()
Insert Statement	
INSERT INTO PKP (user_id, age, status) VALUES ("mongo", 45, "A")	db.PKP.insertOne({ user_id: "mongo", age: 18, status: "A" })
Select Command	
SELECT *FROM PKP	db.PKP.find()

SELECT id, user_id, status FROM PKP	db.PKP.find({ }, { user_id: 1, status: 1 })
SELECT * FROM PKP WHERE status != "A"	db.PKP.find({ status: { $ne: "A" } })
SELECT * FROM PKP WHERE status = "A"	db.PKP.find({ $or: [{ status: "A" } , { age: 50 }] })
SELECT * FROM PKP WHERE age > 25	db.PKP.find({ age: { $gt: 25 } })
SELECT * FROM PKP WHERE user_id like "%bc%"	db.PKP.find({ user_id: { $regex: /bc/ } })
SELECT * FROM PKP WHERE status = "A" ORDER BY user_id ASC	db. PKP. find({ status: "A" }). sort({ user_id: 1 })
SELECT * FROM PKP LIMIT 5 OFFSET 10	db. PKP.find(). limit(5). skip(10)
SELECT * FROM PKP WHERE name NOT IN ('aaa','abc','bbb');	db.PKP.find({ name: { $nin: [‘aaa’,’abc’,’bbb’] } })
Update Statements	
UPDATE PKP SET status = "C" WHERE age > 25	db.PKP.updateMany({ age: { $gt: 25 } }, { $set: { status: "C" } })
UPDATE PKP SET age = age + 3 WHERE status = "A"	db.PKP.updateMany({ status: "A" } , { $inc: { age: 3 }
Delete Statements	
DELETE FROM PKP WHERE status = "D"	db.PKP.deleteMany({ status: "D" })

Aggregation

Aggregation is used to analyze data and produce meaningful results from multiple documents.

It works in a pipeline:
Collection → $match → $group → $sort → $project → Result

Stage	Purpose	SQL Equivalent	Syntax
$match	Filter documents	WHERE	{$match:{condition}}
$group	Group & calculate	GROUP BY	{$group:{_id:"$field", result:{operation}}}
$sort	Sort records	ORDER BY	{$sort:{field:1/-1}}
$project	Select fields	SELECT	{$project:{field:1,_id:0}}

Complete Pipeline Example

Using the Aggregation Framework, perform the following operations:

1. Group students based on their city.
2. Calculate the average marks of students in each city.
3. Sort the result in descending order of average marks.
4. Display only the city name and the average marks.

```
db.students.aggregate([
{
 $group:{
   _id:"$city",
   avgMarks:{$avg:"$marks"}
 }
},
{$sort:{avgMarks:-1}},
{
 $project:{
   City:"$_id",
   AverageMarks:"$avgMarks",
   _id:0
 }
}
])
```

Output

{ City:"Ahmedabad", AverageMarks:87.5 }
{ City:"Surat", AverageMarks:75 }

Example:

Using the Aggregation Framework, perform the following operations:

1. **Select students whose marks are greater than or equal to 80.**
2. **Group the selected students by city.**
3. **Calculate the total marks obtained by students in each city.**
4. **Sort the result in descending order of total marks.**
5. **Display only the city name and total marks.**

Query

```
db.students.aggregate([
{
  $match:{marks:{$gte:80}}
},
{
  $group:{
    _id:"$city",
    totalMarks:{$sum:"$marks"}
  }
},
{$sort:{totalMarks:-1}},
{
  $project:{
    City:"$_id",
    TotalMarks:"$totalMarks",
    _id:0
  }
}
])
```

Output

```
[
  { City: 'Ahmedabad', TotalMarks: 175 },
  { City: 'Rajkot', TotalMarks: 87 },
  { City: 'Surat', TotalMarks: 80 }
]
```

Indexing in MongoDB

MongoDB Indexing Syntax Summary

Category	Command Syntax
Create Single Field Index	db.collection.createIndex({ field: 1 }) (1 for ascending, -1 for descending)
Create Compound Index	db.collection.createIndex({ field1: 1, field2: -1 })
Create Partial Index	db.collection.createIndex({ field: 1 }, { partialFilterExpression: { field: { $gt: value } } })
Default Index (_id)	Automatically created on _id. No command needed. Cannot be dropped.
Get All Indexes	db.collection.getIndexes()
Drop a Specific Index	db.collection.dropIndex({ field: 1 }) or db.collection.dropIndex("field_1")
Drop All Indexes (except _id)	db.collection.dropIndexes()
Find Data Using Index	db.collection.find({ field: value })
Explain Query Execution	db.collection.find({ field: value }).explain("executionStats")
Check Query Plan	explain("executionStats") shows: • stage: "IXSCAN" for index scan • stage: "COLLSCAN" for collection scan
Sort Using Index	db.collection.find().sort({ field: 1 })
Compound Index Prefix Support	Only supports queries that start with the first field in the index.

Examples

1: Single + Compound Index

db.student.createIndex({ age: 1, name: 1 });

db.student.find({ age: { $gt: 15 }, name: "DDD" }).explain("executionStats");

Output: IXSCAN, nReturned: 1, docsExamined: 1

2: Partial Index

db.student.createIndex({ age: 1 }, { partialFilterExpression: { age: { $gt: 30 } } });

db.student.find({ age: { $gt: 30 } }).explain("executionStats");

Output: IXSCAN (on documents age > 30 only)

3: Index on name, Full Collection Scan

db.student.createIndex({ name: 1 });

db.student.find({}).explain("executionStats");

Output: Likely COLLSCAN if fetching all documents

Connection with NodeJS

Example-1
Add name, Surname,Age,Active and date field in persons collection inside Test database. (Select an appropriate Type as per need)
// Type:1 To insert one by one data
// Type:2 Raise exception and catch it, if any problem occurs. Print a proper message for error handling.
// Type : 3 *Array of Object* *(Recommended)*

```
const mg=require("mongoose")
mg.connect("mongodb://127.0.0.1:27017/test")
.then(()=>{console.log("success")})
.catch((err)=>{console.error(err)});

const mySchema=new mg.Schema({
        name:{type:String, required:true},
        Surname:String,
        age:Number
    });

mg.pluralize(null)
```

```
const person=new mg.model("person",mySchema)


                    // Type:1 To insert one by one data
const personData=new person({ name:"ABC",  Surname:"Patel",   age:30})
personData.save() // also returns promise


                    //OR -  Type:2
const createdoc=async()=>{
try
   {
      const persondata=new person({name:"ABC ",Surname:"Patel ",age:30})
      const result=await persondata.save();
      console.log(result)
   }
catch(err){console.log("There is error")}
}
createdoc()

                    //OR - Type:3 Array of Object (Recommended)

const createDocArray=async()=>    {
      try{
      const personData=[{ name:"ABC",  Surname:"Patel",   age:30},
      {name:"XYZ",  Surname:"Patel",   age:30}]
      const result= await person.insertMany(personData)
      console.log(result)       }
         catch(err){
            console.log("problem")}
   }
createDocArray();
```

Example -2 [find sample example using push() metod..]
Write a node.js script to enter more than one document to the collection after establishing a connection using mongoose. Raise exception and catch it, if any problem occurs. Print a proper message for error handling. Else find some records.

```
const mg=require("mongoose")
mg.connect("mongodb://127.0.0.1:27017/9_7").then(()=>{console.log("success")}
).catch((err)=>{console.error(err)});
//mg.pluralize(null)
const mySchema=new mg.Schema({
      name:{type:String, required:true },
      Surname:String,  age:Number,   active:Boolean,  date:{ type:Date, default:new Date()}
   });
const person=new mg.model("person",mySchema)
const createDoc=async()=> {
      try{
         const personData1=[
         {name:"test", Surname:"test1", age:33, active:true},
         {name:"hi", Surname:"hi1", age:30, active:true},
         {name:"hello", Surname:"hello1", age:37, active:true},
         {name:"hello",Surname:"hello11", age:37, active:true}]

 const result= []
    result.push(await person.insertMany(personData1))
    result.push(await person.find({name:"abc"},{date:1,_id:0}))
    result.push(await person.find({name:"hi"}).limit(1))
    result.push(await person.find({name:"hello1"}).select({name:1}).limit(1))
    result.push(await person.findOne({name:"abc"}).select({Surname:1}))

    console.log(result)
      }
         catch(err) { console.log("problem"); }
   }
createDoc();
```

Example:

To demonstrate update/delete record by using three different methods.

- **updateOne**
- **findByIdAndUpdate**
- **findByIdAndDelete**

```
const mg=require("mongoose");
mg.connect("mongodb://127.0.0.1:27017/lju1");
const personSchema=new mg.Schema({
   name:String,
   age:Number,
   active:Boolean
});
const Person=mg.model("Person",personSchema);
const performOperations=async()=>{
   try{

      //Insert a document
      const personData=new Person({name:"test",age:25,active:true});
      await personData.save();

      //Update one document
      const result=await
Person.updateOne({name:"test"},{$set:{age:34,active:false}},{upsert:true});
      console.log("Update Result:",result);

      //Find one document
      const person=await Person.findOne({name:"test"});
      console.log("Person:",person);
      console.log("Person ID:",person._id);

      //Update using _id
      const updatedPerson=await
Person.findByIdAndUpdate(person._id,{name:"LJU",age:28,active:true},{new:true});
//Returns the updated document after the update.
      console.log("Updated Person:",updatedPerson);

      //Delete using _id
      const deletedPerson=await Person.findByIdAndDelete(person._id);
      if(deletedPerson)
         console.log("Deleted Person:",deletedPerson);
      else
         console.log("Person not found");

   }
   catch(err){console.error(err);}
};
performOperations();
```

Methods in mongoose

Method	Match by	Action	Returns Result?	Common Use Case
updateOne()	Filter	Update	Result object	Update one doc with filter
updateMany()	Filter	Update many	Result object	Bulk update
findByIdAndUpdate()	_id	Update	Document	Update by ID, get doc
findOneAndUpdate()	Filter	Update	Document	Custom match, get updated doc
findByIdAndDelete()	_id	Delete	Deleted doc	Delete by ID
findOneAndDelete()	Filter	Delete	Deleted doc	Delete with custom match
deleteOne()	Filter	Delete one	Delete result	Simple delete
deleteMany()	Filter	Delete all	Delete result	Mass delete
save()	New/Existing	Insert/Update	Saved doc	Create or update a document instance

Mongoose Schema validation

Validator / Option	Purpose
required:`true`	Makes a field mandatory
unique:`true`	Prevents duplicate values
default:`value`	Assigns a default value
min, max	Sets minimum and maximum values
minlength, maxlength	Sets minimum and maximum string length
enum:[`value1,value2]`	Allows only specified values
match:`/pattern/`	Validates using a regular expression
trim:`true`	Removes leading and trailing spaces
lowercase:`true`	Converts text to lowercase
uppercase:true	Converts text to uppercase
validate: v.isEmail etc	Defines a custom validation function
strict	Allows or restricts fields outside the schema

strict option

The **strict** option controls whether fields that are **not defined in the schema** are stored.

strict:true **(Default)**	strict:false
Stores only schema fields	Stores schema fields and extra fields
Extra fields are ignored	Extra fields are also saved

Example:

Define a Mongoose schema called userSchema with the following fields and validation requirements:

1. username:
    a. Required and must be between 4 and 20 characters long.
    b. Must start with letters and end with digits. (Ref*)
    c. Should be trimmed of any leading or trailing spaces.
    d. Should be converted to uppercase before saving.
2. email:
    a. Required, must be unique across the collection. **(using validator module)**
    b. Must follow the standard email format.
3. age:
    a. Must be a number between 18 and 65.
4. role:
    a. Must be either 'user' or 'admin'.
    b. Should default to 'user' if not provided.

```
const mg = require('mongoose');
const v = require('validator');

mg.connect("mongodb://127.0.0.1:27017/validations").then(()=>{console.log("success")}).cat
ch((err)=>{console.error(err)});
const userSchema = new mg.Schema({
  username: {
    type: String,
    required: [true, 'Username is required'], // Custom error message
    minlength: [4, 'Username must be at least 4 characters long'],
    maxlength: [20, 'Username cannot be more than 20 characters long'],
  //  match:[/^[A-Za-z]+[0-9]+$/,"Must start with letters and end with digits"],
    trim:true,
    uppercase:true
  },
  email: {
```

```
    type: String,
    unique: [true, 'email already exists'],
    required: [true, 'Email is required'],
    validate: [ v.isEmail, `This is not a valid email address!`]

},
  age: {
    type: Number,
    min: [18, 'Age must be at least 18'],
    max: [65, 'Age must be less than 65']
  },

  role: {
    type: String,
    enum: ['user', 'admin'], // Value must be either 'user' or 'admin'
    default: 'user'
  }
});
```

Connectivity of MongoDB with ExpressJS using HTML

Example: Create a form having username and password and insert data entered by user in collection named "data1" in mongoDB.

In task.js file

```
var expr=require("express")
var app=expr()
const mg=require("mongoose")

mg.connect("mongodb://127.0.0.1:27017/login")
.then(()=>{console.log("Successful")})
.catch((err)=>{console.error(err)})

mg.pluralize(null)
const myschema=new mg.Schema({
        uname:{type:String, required:true},
        password: {type:String, required:true}
})
const person =new mg.model("data1", myschema)

app.use(expr.static(__dirname,{index:"form.html"}))
```

```
app.get("/process_get",(req,res)=>{
      const personData=new person({
      uname:req.query.uname,
      password:req.query.pwd})
personData.save()
res.send("Record inserted")
})
app.listen(6000)
```

form.html

```
<html>
   <form action="/process_get" method="get">
      Username: <input type="text" name="uname"/>
   </br> Password: <input type="password" name="pwd"/>
   </br> <input type="submit"/>
   </form>
</html>
```

Connectivity of MongoDB with React

Signup.jsx

```
import React, { useState } from 'react';
import axios from 'axios';

function Signup() {
  const [username, setUsername] = useState('');

  const handleSignup = async (e) => {
    e.preventDefault();

    try {
      await axios.post('http://localhost:5000/signup', {username});
     alert('User signed up successfully.'+username);
     setUsername('');
     }
 catch (error) {
      console.error('Error signing up:', error);
     alert('An error occurred.');
    }
  };
  return (
```

```
   <div>
     <h1>Sign Up</h1>
     <form onSubmit={handleSignup} method =’post’>
     Name:  <input  type="text"  placeholder="Username"  value={username}
        onChange={(e) => setUsername(e.target.value)} />
       <button type="submit">Sign Up</button>
     </form>
   </div>  )}

export default Signup;
```

server.js

```
const express = require('express');
const mg = require('mongoose');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json());
mg.connect('mongodb://127.0.0.1:27017/User') .then(()=>{console.log("Connection
Success")})
const UserSchema = new mg.Schema({  username:String  });

const User = new mg.model('User', UserSchema);

app.post('/signup', async (req, res) => {
  try {
    const { username} = req.body;
//console.log(“Username is” + req.body.username)

    const newUser = new User({ username});
    await newUser.save();
    res.send();
// or res.json({message:’Username inserted Successfully’})
  } catch (error) {
    res.send(error);
//or res.json({message:’Error in inserted Record’})
  }
});
app.listen(5000);
```